KleverOne Planning Agent — Architecture (Sprint 1)

Status: Draft for engineering hand-off · **Audience:** 1 full-stack engineer + 1 AI engineer + PO · **Last updated:** 2026-05-12
Source consultations: solution-architect, ai-engineer, fullstack-engineer, ux-designer, ui-designer, qa-engineer, product-manager
Companion docs: `PRD.md`, `submission-v2.pdf`, `context.txt`

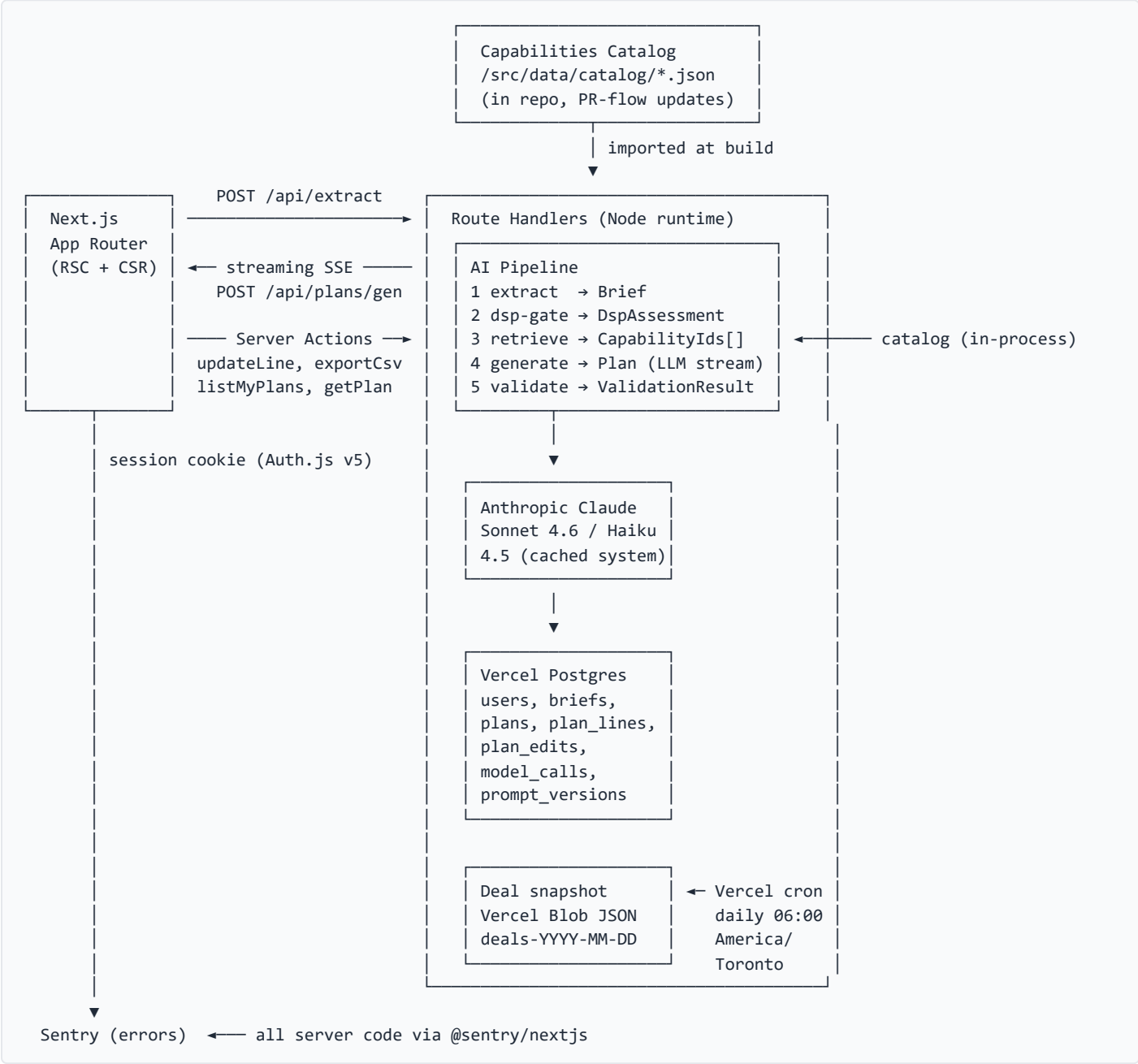
0. Executive Summary

A Next.js 16 App Router on Vercel, fronting a deterministic-first AI pipeline (Anthropic Claude Sonnet 4.6 + Haiku 4.5), persisting plans, briefs, edits, and a full model-call log to Vercel Postgres. The product's cultural job — make SSP-direct the default and DSP the defended exception — is enforced by **three reinforcing layers**:

1. A **deterministic DSP gate** before any LLM call decides which (if any) DSP vendors are even eligible.
2. A **structured output contract** that makes `dsp_justification` mandatory on every DSP line and capability/deal references non-hallucinable.
3. A **regression harness in CI** with hard gates on schema, hallucination, DSP share, and justification — running on every prompt or model PR.

The frontend renders the plan as a dense, scannable table where DSP lines carry a dedicated always-visible justification column (the cultural mechanic must survive a screenshot, which row tints don't). Edits are inline, optimistic, captured as structured telemetry — the highest-fidelity feedback the platform will ever get. Sprint 1 defers live MCP calls, AdCP integration, GAM export, and SSO. What ships is end-to-end and protected.

1. System Diagram



Happy path: brief text → `/api/extract` (Sonnet, 1–2s) → editable `Brief` → user confirms → `/api/plans/generate` streams `DspAssessment` then plan lines (Sonnet, 10–18s) → post-validation → persist → client renders streamed plan and enters edit mode where each edit fires a Server Action writing to `plan_edits`.

2. Architecture Principles

1. **Deterministic before LLM.** Anything reducible to a filter, rule, or lookup is deterministic. LLM tokens on closed-form problems import non-determinism, latency, and cost for no recall gain.
2. **Structured output is the contract.** The plan schema is the single source of truth across AI engineer, full-stack engineer, and QA. Locked week one; every diff after pays rework.
3. **Audit everything once.** Every plan reconstructible from `brief_snapshot`, `prompt_version_id`, `catalog_version`, `deal_snapshot_date`. Half a day in v1; impossible to retrofit in v3.

4. **Plan edits are signal.** Inline, optimistic, structured event per change. Painful editing kills the team's feedback loop and the cultural mechanic regresses unseen.
5. **DSP visibility is a UI requirement, not a model requirement.** A model that picked the right answer is wasted if the UI buries the `dsp_justification`.
6. **Sprint 1 ships the eval with the product.** Without it the cultural mechanic regresses on the first prompt tweak.

PART A — BACKEND & AI PIPELINE

3. AI Pipeline (5 Stages)

Single in-process composition inside `/src/lib/pipeline/`. No queue, no worker — Vercel function timeout on Pro is 300s, well above the 30s budget. Anthropic SDK only with prompt-caching beta header if not GA by build time.

Stage	Input	Output	Kind	Model	Cache scope	Latency	Failure
1. Extraction	Free-text brief (≤4k chars)	`Brief` (11 fields)	LLM, tool-use forced	`claude-sonnet-4-6`	system + field schema (~800 tok)	**2.0s p95**	Schema fail → 1 retry with errors injected; second pass → partial Brief with `_unknown` flags
2. DSP gate	`Brief`	`DspAssessment`	**Deterministic**	none	n/a	<5ms	Cannot fail; total over brief domain
3. Retrieval	Brief + eligible vendors	filtered `Capability[]`	**Deterministic** filter on channels∩scope, kpi, vendor-type	none	n/a	<10ms	Empty subset → abort with `RETRIEVAL_EMPTY`
4. Plan generation	Brief, assessment, capabilities, deal subset	`Plan` (streamed)	LLM, tool-use forced, tool `emit_plan`	`claude-sonnet-4-6`	system + full catalog (~8–12k tok)	**12–18s p95**, first token ≤5s	Validation fail → 1 retry with errors injected; second → `VALIDATION_FAILED`
5. Post-validation	Plan	`ValidationResult`	**Deterministic**	none	n/a	<50ms	On success, write Plan + lines + ModelCall in single transaction

Total latency target: under 25s p95, first plan line visible under 6s.

Stage-2 rules (deterministic DSP gate)

- **DV360 eligible iff** `brief.channelsInScope` includes `"youtube"`. Reason: YouTube walled-garden inventory.
- **TTD eligible iff** brief has a retargeting-pool signal (audience mentions retargeting / site visitors / cart abandoners) **OR** `kpi ∈ {CPA, CTR}` AND `goal === "performance"`.
- **Amazon DSP eligible iff** `vertical ∈ retail-endemic-list` AND `kpi ∈ {CPA, CTR}`.
- Otherwise all DSPs rejected with reasons attached. Each eligible vendor carries `share_ceiling` (DV360 25%, TTD 35%, Amazon DSP 35%) and `share_ceiling_reason`.
- The LLM cannot introduce a DSP line absent from `eligible[]`; the validator rejects.

Why these choices

- **Sonnet 4.6, not Opus, for generation:** the cultural job is enforced by the gate + schema. Opus is ~5× cost for marginal quality. Promote only if eval shows Sonnet regressing on rationale judges.
- **Structured filter, not vector search for stage 3:** catalog is closed-vocabulary, ≤200 entries, with explicit channels/kpis. Embeddings add complexity for zero recall gain.
- **Haiku 4.5 held in reserve.** The only obvious Haiku target is explicitly deferred per PRD §3.
- **Prompt caching:** system prompt + full catalog in one `cache_control: { type: 'ephemeral' }` block. ~85% hit rate within a session. Per-call cost ~$0.06 → ~$0.02 once warm.

4. Data Layer

4.1 Postgres tables (Vercel Postgres, schema `app`)

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE app.users (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email           text UNIQUE NOT NULL,
  display_name    text,
  created_at      timestamptz NOT NULL DEFAULT now(),
  last_login_at   timestamptz
);

CREATE TABLE app.prompt_versions (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  stage           text NOT NULL CHECK (stage IN ('extract','generate')),
  semver          text NOT NULL,
  git_sha         text NOT NULL,
  prompt_hash     text NOT NULL,
  model_id        text NOT NULL,
  notes           text,
  created_at      timestamptz NOT NULL DEFAULT now(),
  UNIQUE (stage, semver)
);

CREATE TABLE app.briefs (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         uuid NOT NULL REFERENCES app.users(id),
  raw_text        text NOT NULL,
  extracted       jsonb NOT NULL,
  extract_prompt_version_id uuid NOT NULL REFERENCES app.prompt_versions(id),
  edits           jsonb NOT NULL DEFAULT '[]'::jsonb,
  created_at      timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE app.plans (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         uuid NOT NULL REFERENCES app.users(id),
  brief_id        uuid NOT NULL REFERENCES app.briefs(id),
  brief_snapshot  jsonb NOT NULL,
  dsp_assessment  jsonb NOT NULL,
  total_budget    integer NOT NULL,
  generated_at    timestamptz NOT NULL DEFAULT now(),
  generate_prompt_version_id uuid NOT NULL REFERENCES app.prompt_versions(id),
  catalog_version text NOT NULL,
  deal_snapshot_date date NOT NULL,
  status          text NOT NULL DEFAULT 'draft'
                  CHECK (status IN ('draft','exported','archived'))
);
CREATE INDEX plans_user_id_generated_at_idx ON app.plans(user_id, generated_at DESC);

CREATE TABLE app.plan_lines (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  plan_id         uuid NOT NULL REFERENCES app.plans(id) ON DELETE CASCADE,
  ordinal         integer NOT NULL,
  vendor          text NOT NULL,
  vendor_type     text NOT NULL CHECK (vendor_type IN ('SSP','DSP','AdCP')),
  channel         text NOT NULL,
  path            text NOT NULL CHECK (path IN ('ssp_direct','adcp','dsp')),
  spend_pct       numeric(5,2) NOT NULL,
  spend_dollars   integer NOT NULL,
```

```
  deal_refs       text[] NOT NULL DEFAULT '{}',
  capability_ids  text[] NOT NULL DEFAULT '{}',
  rationale       text NOT NULL,
  allocation_rationale text NOT NULL,
  dsp_justification text,
  share_ceiling   numeric(5,2),
  share_ceiling_reason text,
  CONSTRAINT dsp_fields CHECK (
    (vendor_type = 'DSP'  AND dsp_justification IS NOT NULL AND share_ceiling IS NOT NULL)
    OR (vendor_type <> 'DSP' AND dsp_justification IS NULL AND share_ceiling IS NULL)
  )
);
CREATE INDEX plan_lines_plan_id_ordinal_idx ON app.plan_lines(plan_id, ordinal);

CREATE TABLE app.plan_edits (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  plan_id         uuid NOT NULL REFERENCES app.plans(id) ON DELETE CASCADE,
  line_id         uuid REFERENCES app.plan_lines(id) ON DELETE CASCADE,
  user_id         uuid NOT NULL REFERENCES app.users(id),
  field           text NOT NULL,
  old_value       jsonb NOT NULL,
  new_value       jsonb NOT NULL,
  edited_at       timestamptz NOT NULL DEFAULT now(),
  exceeds_ceiling boolean NOT NULL DEFAULT false
);

CREATE TABLE app.model_calls (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         uuid REFERENCES app.users(id),
  plan_id         uuid REFERENCES app.plans(id) ON DELETE SET NULL,
  brief_id        uuid REFERENCES app.briefs(id) ON DELETE SET NULL,
  stage           text NOT NULL,
  prompt_version_id uuid NOT NULL REFERENCES app.prompt_versions(id),
  model_id        text NOT NULL,
  request         jsonb NOT NULL,
  response        jsonb NOT NULL,
  input_tokens    integer NOT NULL,
  output_tokens   integer NOT NULL,
  cache_read_tokens  integer NOT NULL DEFAULT 0,
  cache_write_tokens integer NOT NULL DEFAULT 0,
  cost_usd        numeric(10,6) NOT NULL,
  latency_ms      integer NOT NULL,
  validation_ok   boolean,
  validation_errors jsonb,
  retry_of_id     uuid REFERENCES app.model_calls(id),
  is_eval_run     boolean NOT NULL DEFAULT false,
  run_id          text,
  created_at      timestamptz NOT NULL DEFAULT now()
);
```

Audit columns: `briefs.raw_text`, `briefs.extracted`, `plans.brief_snapshot`, `plans.dsp_assessment`, `plans.catalog_version`, `plans.deal_snapshot_date`, all of `plan_edits`, all of `model_calls`. `brief_snapshot` is intentionally denormalized — a user editing their brief after generation must not mutate the plan's input record. Provide `getPlanInputBrief(planId)` helper; ban direct `plans → briefs` joins for input reads.

ORM: **Drizzle** (Vercel Postgres first-class).

4.2 Capabilities catalog

- **Location:** `/src/data/catalog/capabilities.json`.
- **Schema:** the `Capability` type from `types.ts`.

- **Update flow:** PR to `main`. Required reviewer: catalog owner (PRD #2). CI validates shape + runs regression suite. Merge → redeploy; `plans.catalog_version = git sha`.
- **Cutover trigger:** first time a Strategist reports a stale citation in production, **or** a second capability change ships within 7 days. Promote to Airtable/Notion-as-DB with 5-min cache. Don't pre-build.

4.3 Deal snapshot

- **Format:** `{ snapshotDate, deals: { [vendor]: DealRef[] } }`.
- **Storage:** Vercel Blob, key `deal-snapshots/YYYY-MM-DD.json`. Public-read, authed writes.
- **Refresh:** Vercel Cron `0 10 * * *` UTC, calls `/api/cron/refresh-deals`. Sprint 1 reads `/src/data/deal-snapshot-seed.json`; replacing with live MCP calls is sprint 2+.
- **Naming:** `planning-deal-snapshot-*` so buying agent's freshness need is an explicit handoff problem.

5. API Surface

Convention: **Server Actions** for mutations triggered by a form/button where streaming isn't needed; **Route Handlers** for streaming, cron, and anything outside RSC.

Surface	Kind	Method/path	Request	Response	Stream
Extract brief	Route Handler	`POST /api/extract`	`{ rawText }`	`{ brief, briefId }`	No
Generate plan	Route Handler	`POST /api/plans/generate`	`{ briefId, brief }`	SSE: `dsp_assessment`, `line`, `validation`, `plan`	**Yes**
Update plan line	Server Action	`updatePlanLine(...)`	patch	`{ plan, edits }`	No
Export CSV	Route Handler	`GET /api/plans/:id/export.csv`	path	`text/csv`	No
Export markdown	Server Action	`exportPlanMarkdown`	`{ planId }`	`{ markdown }`	No
List my plans	Server Action	`listMyPlans()`	—	`PlanSummary[]`	No
Get plan	Server Action	`getPlan(planId)`	`{ planId }`	`{ plan, edits }`	No
Cron: refresh deals	Route Handler	`GET /api/cron/refresh-deals`	cron header	`{ snapshotDate }`	No

CSV column order (locked for AdOps): `ordinal, vendor, vendor_type, path, channel, spend_pct, spend_dollars, deal_refs, capability_ids, rationale, allocation_rationale, dsp_justification, share_ceiling, share_ceiling_reason`. Reordering after sprint 1 is a breaking change.

Streaming: Anthropic SDK `messages.stream()` with `tool_use` produces partial JSON. Server parses incrementally (`partial-json` package, ~3kb) and emits an SSE event per fully-formed plan line. Final `event: plan` after validation. Retries server-side, transparent to client.

Auth on both surfaces: Next.js 16 docs warn Server Functions are reachable by direct POST. Both call `await requireUser()` at top — implement once in `/src/lib/auth/session.ts`.

PART B — FRONTEND ARCHITECTURE

6. Route Map (Next.js App Router)

```
app/
  layout.tsx                          [server]  AuthShell, fonts, Sentry init
  page.tsx                            [server]  redirect → /plans
  plans/
    layout.tsx                        [server]  authed shell, nav, user chip
    page.tsx                          [server]  list — server-fetched
    new/
      page.tsx                        [server]  brief intake, hosts <BriefInput/>
    [id]/
      page.tsx                        [server]  fetch plan + brief, mount <PlanWorkspace/>
      not-found.tsx                   [server]
      error.tsx                       [client]  segment error boundary
      loading.tsx                     [server]  skeleton shell
  api/
    plans/[id]/stream/route.ts        [server]  GET, streams plan
  actions/
    plans.ts                          [server]  create, updateBrief, patchLine, logEdit, exportCsv
```

- **`/plans` is a route.** Strategists return to past plans; sidebar is sprint-2.
- **`/plans/new` is a route.** Back button works; refresh doesn't nuke a paste.
- **`/plans/[id]`** carries confirmation banner + table + assessment + export. Confirmation is an **inline section**, not a separate route.
- **No `/plans/[id]/edit`.** Edit is inline; separating would regress edit-as-feedback.
- **API route only for streaming.** Everything else through Server Actions.

7. Component Tree

```
<AuthShell>                                          (server)
  <SiteHeader/>                                      (server)  mark, user chip, "New plan"
  <main>
    /plans               → <PlansList/>              (server)
    /plans/new           → <BriefInput/>             (client)
    /plans/[id]          → <PlanWorkspace/>          (client)
        <PlanWorkspaceHeader/>                       (client)
        <ExtractedBriefConfirmation/>                (client)
        <DspGateSummary/>                            (client)
        <EditsCapturedIndicator/>                    (client)
        <PlanTable/>                                 (client)
            <PlanLineRow/> × N                       (client)
            <StreamingSkeletonRow/> × (expected − N) (client)
            <PlanTableFooter/>                       (client)
        <ValidationIssues/>                          (client)
  <Toaster/>                                         (client)  shadcn sonner
```

Component	S/C	Owns
`AuthShell`	server	auth gate, allowlist redirect
`BriefInput`	client	textarea, "Use example", submit
`ExtractedBriefConfirmation`	client	11-field form (RHF + zod), per-field edit capture
`PlanWorkspace`	client	URL→state hydration, edit-count diff, stream subscription
`DspGateSummary`	client	one-line summary; expand-to-read rejections
`EditsCapturedIndicator`	client	pill, count
`PlanTable`	client	colgroup, header, footer, row iteration
`PlanLineRow`	client	one `<tr>`, ceiling warning, edit acks
`StreamingSkeletonRow`	client	skeleton `<td>`s matching colgroup
`ExportMenu`	client	Export CSV, Copy markdown
`ErrorBoundary`	client	recoverable error UI

8. The DSP-Line Visual Treatment Decision

Decision: dedicated always-visible justification column, populated and prominently styled only on DSP lines.

Defense vs row-tint: tint requires the eye to learn "this color = read column X"; a screenshot pasted into a slide deck strips that signal. The cultural mechanic must travel with the row — color does not.

Defense vs vendor-cell badge + truncated justification: collapsing forces a tooltip or expand to read — the "buried in expand-to-read" failure mode. AdOps pasting into Sheets loses badge styling.

Absence of justification on SSP/AdCP lines becomes a silent em-dash — the right default. DSP rows get a 2px **left border on the justification cell only** in functional amber (`border-amber-500`). Survives screenshot, accessibility (text + shape, not color-only), and is implementable in shadcn primitives in under an hour.

9. Table Design

#	Column	Align	Type	Max	Truncation
1	Vendor	left	prose, semibold	120px	none
2	Type	left	badge	64px	n/a
3	Channel	left	prose	88px	none
4	Spend %	**right**	tabular num, editable	96px	n/a
5	Spend $	**right**	tabular num, computed	112px	n/a
6	Deal / PMP	left	mono chips	168px	wrap
7	Capabilities	left	prose list, stacked	168px	name only; full on `title`
8	Rationale	left	prose, editable	240px	`line-clamp-2`, expand
9	Allocation rationale	left	prose, editable	220px	`line-clamp-2`, expand
10	DSP justification	left	prose	260px	`line-clamp-3`, expand (read-only)

Total width ~1356px. Fits 1280px with light scroll; breathes at 1440px+.

Type stack: Inter Variable (sans, with `tabular-nums`), JetBrains Mono Variable (capability/deal IDs).

Edit affordances: dotted underline at rest → solid on hover (cursor-text) → 1px border + inline textarea on focus → post-edit solid underline + `font-medium` permanently. Non-editable cells: no underline, no cursor change. Workspace-level "edits captured" pill is the global ack; per-row ticks add noise.

Density target: 8–10 lines visible on 1080p without scroll. Row 56–64px. Header 36px, footer 36px, page chrome ~140px → 904px available → 14 rows at 64px.

Screenshot survival: table must look intentional cropped to cols 1–5 and cols 1, 2, 10 — the two screenshots a Strategist will paste. No row-level tints, no decorative gridlines.

10. State Management

Concern	Lives in	Why
Brief text (pre-extraction)	client state in `<BriefInput/>`	ephemeral; refresh loss OK
Extracted Brief (form)	react-hook-form seeded from server prop	RHF + zod mirrors AI engineer's schema
Plan (committed)	Vercel Postgres, hydrated as `initialPlan`	URL canonical; refresh restores
Plan (in-flight)	`useState` in workspace, appended via AI SDK hook	partial → state → table
Edits (uncommitted)	`useState<PlanLine[]>` + diff vs `originalLines`	optimistic; persisted per edit
Edit count	derived selector	one source of truth

Original plan stays pinned on the server record (`plans.brief_snapshot`). Edits diff against it — "edits diverging from model output" is the team's signal, not "edits since previous edit."

Edit capture event

```
export type EditEvent = {
  edit_id: string;
  plan_id: string;
  line_id: string | null;
  field: 'spendPct' | 'rationale' | 'allocationRationale' | 'dspJustification';
  before: string | number;
  after: string | number;
  user_id: string;
  ts: string;
  prompt_version: string;
  session_id: string;
};
```

Optimistic update lands instantly. Server Action runs in `startTransition`. Failed write toasts "edit not saved, retry" — does **not** roll back UI. Debounce prose at 600ms on blur; spend % fires on commit (Enter/blur).

11. Streaming UX

Vercel AI SDK `streamObject`; server route at `app/api/plans/[id]/stream/route.ts` returns a stream typed against `Plan.lines[]`. Client uses `useObject({ api, schema })` exposing a partial array that grows as tokens arrive.

First 5 seconds:

1. **0.0s** — User clicks "Generate plan". Server Action writes brief, returns plan id.
2. **0.3s** — `/plans/[id]` mounts. Renders header, collapsed brief banner, table with 8 skeleton rows.

3. **0.5s** — Stream opens. Pulsing dot in footer: "Drafting plan…"
4. **3–8s** — First line resolves: skeleton replaced with `fade-in duration-200`.
5. **5s** — User sees brief banner editable, first 1–2 real lines, rest pending. Can scroll, expand gate summary, or wait.

Skeleton widths match the colgroup exactly so no reflow when rows land. `motion-safe:animate-pulse` only — on `prefers-reduced-motion` the pulse stops; real rows still distinguishable by text content. No shimmer.

Mid-stream error: keep rendered rows; replace remaining skeletons with a full-width row: "Generation stopped at line N of M. [Resume] [Start over]". Sentry captures with `planId`, `linesReceived`.

12. Error and Edge States

State	User sees	Action	Sentry
Empty `/plans`	"No plans yet." + CTA + faded sample plan	Click CTA	none
Empty plan output	"Agent returned zero lines. Brief likely underspecified. [Edit] [Retry]"	Edit/retry	warn `empty_plan_returned`
Model timeout (30s)	Inline "Taking longer than usual…" → at 60s "Timed out. [Try again] [Edit]"	Retry/edit	error `generation_timeout`
Schema mismatch	Red banner: "Invalid plan. Team notified. [Try again]"	Retry	error `schema_validation_failed`
Stream drop	See §11 mid-stream	Resume/restart	warn `stream_aborted`
Edit conflict	Toast: "Edit not saved. [Retry]" — value remains	Retry	warn `edit_persistence_failed`
Export failure	Toast: "Export failed — [Try again] or [Copy markdown]"	Try again	warn `export_failed`
Auth expired	"Session expired. [Sign in]"; return URL preserved	Sign in	none
Not found / not owned	404: "Plan not found. [See your plans]"	Back	warn `plan_not_found`

13. Microcopy (terminology locked)

Surface	Copy
Brief heading	"Paste a campaign brief"
Brief sub	"The agent extracts structured fields, then drafts a media plan with SSP-direct as the default and DSPs only where they earn a seat."
Brief CTA	**"Draft plan"**
Confirmation heading	"Confirm the brief"
Confirmation CTA	**"Generate plan"**
Workspace heading	"Draft plan — {advertiser}"
Edits pill	"{N} edit{s} captured — these become structured feedback for the planning agent."
DSP gate, none earned	"DSP gate · no DSP earned a seat on this plan"
DSP gate, N earned	"DSP gate · {N} DSP{s} earned a seat ({names}), {M} rejected"
Export menu items	"Export CSV (for AdOps)", "Copy as markdown"
Regenerate confirm	"Regenerate will replace your current draft and discard {N} edit{s}. Continue?"
Empty `/plans`	"No plans yet." + "Draft your first plan"
Error template	"We couldn't generate this plan — {reason}. [Try again] [Edit fields and retry]"

Terminology lock: "plan" everywhere. "Draft plan" is the heading qualifier; otherwise "plan."

14. Accessibility Baseline

- **Contrast:** WCAG AA. `zinc-700` on white floor. `amber-900` on `amber-50` for DSP justification cell.
- **Focus:** shadcn 2px ring on every `<input>`, `<button>`, `<textarea>`. Tab order: row by row, left to right, only editable cells; non-editable wrappers use `tabIndex={-1}`.
- **Color is never the only signal:** DSP rows = amber + left border + populated text. Over-ceiling = amber text + triangle glyph + literal "over ceiling of X%."
- **Motion:** `motion-safe:animate-pulse`; reduced motion → static "Generating…" line count.
- **Live regions:** streaming footer `aria-live="polite"`; edits pill `aria-live="polite" aria-atomic="true"`.
- **Esc** collapses expanded rationale textarea without saving; **Enter** on spend % commits and advances focus to next row's spend cell.

PART C — QA, EVALUATION & CI

15. Wrong-Plan Taxonomy → Detection

Class	Detection	Severity	Test class
DSP-default creep	`dsp_share_threshold`: pure-SSP 0%, YouTube 25%, retargeting/retail 40%, mixed 15%	Hard	pure-SSP, retail-perf, edge
Justification absence	`dsp_justification_present_and_nonempty` (regex: ≥40 chars, ≥1 brief token)	Hard	YouTube, retargeting, retail
Justification stubbing	LLM judge `dsp_justification_adequacy`	Soft	same
Capability hallucination	`capability_ids_resolve` lookup	Hard, 0-tol	All
Deal hallucination	`deal_refs_resolve` lookup	Hard, 0-tol	All
Constraint violation	`geo_subset_of_brief`, `inventory_matches_brief`, `exclusions_respected`	Hard	premium, retargeting
KPI/channel mismatch	`kpi_channel_match` lookup table	Hard	edge, retail-perf
Budget errors	`shares_sum_to_100`, `dollars_match_budget`, `no_negative_shares`	Hard	All
Rationale quality	LLM judge pairwise vs reference	Soft	All

16. Golden Brief Set

Count: 20 briefs. Owned in `evals/briefs/`, JSON, PR-versioned.

Category	Count	What's tested
pure-SSP	5	Premium video, no DSP-eligible signal. Expected DSP share = 0%.
YouTube-in-scope	4	Forces justified DV360. DSP share 10–25%, justification cites YouTube uniqueness.
retargeting-performance	3	Retargeting pool + CPA. DSP earns 25–40%.
retail-performance	3	Amazon retail, CPA, Amazon DSP eligible. 20–40%.
underspecified-edge	3	Missing KPI or geo. Extractor flags; generation blocked until confirmed.
contradictory	2	Premium + sub-$5 CPM, or "no UGC" + TikTok. Plan respects hard constraint, surfaces tension.

```
evals/
  briefs/
  expected/
  fixtures/
    catalog-snapshot.json
    deals-snapshot.json
  judges/
    rationale_quality.v1.md
    allocation_rationale_quality.v1.md
    dsp_justification_adequacy.v1.md
  reference-prompt-version.txt
  manifest.json
```

Sign-off flow: Named Strategist (PRD #1, ~4 hrs/week) reviews each reference in a PR. CODEOWNERS rule on `evals/expected/**` forces their approval. Quarterly re-validation, or on model swap. **Without a named human, the eval cannot ship.**

17. Deterministic Checks

Check	Pass rule	Severity
`schema_conforms`	Zod parse against locked plan schema	Hard
`shares_sum_to_100`	|sum − 100| ≤ 0.5	Hard
`dollars_match_budget`	|sum − budget| ≤ $1	Hard
`no_negative_shares`	All ∈ [0, 100]	Hard
`capability_ids_resolve`	Every id in catalog	Hard, 0-tol
`deal_refs_resolve`	Every ref in snapshot	Hard, 0-tol
`dsp_justification_present_and_nonempty`	DSP lines: ≥40 chars, ≥1 brief token	Hard
`non_dsp_lines_no_justification`	Non-DSP: `dspJustification === null`	Hard
`dsp_share_threshold`	≤ category threshold	Hard
`geo_subset_of_brief`	Plan geo ⊂ `brief.geo`	Hard
`inventory_matches_brief`	Premium → no `open-exchange` capabilities	Hard
`kpi_channel_match`	Static KPI→channel lookup	Hard
`share_ceiling_respected`	DSP `spendPct ≤ shareCeiling`	Hard
`dsp_gate_summary_consistent`	Lines and `dspAssessment` agree	Hard
`required_capabilities_present`	≥1 of `requiredCapabilityIds`	Soft
`channel_mix_within_range`	Each share within ±10pp of reference range	Soft

18. LLM-Judge Harness

Judge model: Claude Haiku 4.5 (`claude-haiku-4-5-20251001`). Cheaper, faster, and weak enough that it won't paper over generator failures.

Pairwise vs absolute: Pairwise against reference. Absolute drifts ~0.5 points/month on a 1–5 rubric; pairwise stays stable because both sides move together. Matches how the named Strategist will calibrate.

Three judges per plan — `rationale_quality`, `allocation_rationale_quality`, `dsp_justification_adequacy` (last only on DSP plans). Each emits `{winner, confidence, reasons}`. Win rate aggregated across 20 briefs.

Rubric — `dsp_justification_adequacy` (the critical one)

```
Compare two dsp_justification strings on the same DSP line.

Prefer the one that explicitly covers all three:
- What's unique to this DSP (inventory, data, signal SSP/AdCP cannot match).
- What SSP-direct or AdCP CANNOT do for this brief — name the counterfactual.
- Why this share, not larger.

Penalize:
- Asserts uniqueness without naming the source.
- Recycles the line rationale.
- Under 40 or over 120 words.

HARD FAIL (winner = "neither", confidence = 5) if SSP/AdCP counterfactual not named.

Output: {winner, confidence, reasons}.
```

Calibration: once per sprint and on every judge prompt change, the named Strategist labels 30 plan pairs blind. **Cohen's kappa ≥ 0.6** per rubric; below that the rubric is revised. Monthly re-run; alert on drop > 0.1. Judge model pinned to specific version. Re-calibrate on every judge model bump.

19. Edit-as-Signal Capture (Online Eval)

Metric	Definition	Threshold / use
`edits_per_plan_p50`	Median edits per accepted plan	PRD trust metric: ≤3
`spend_pct_edit_magnitude_p50`	Median |Δ| on spendPct edits	Allocation drift indicator
`regenerate_rate`	Plans with ≥2 generations per session	Plan unusable as draft
`time_to_accept_p50`	First-generate → first-export	Coordinate with latency budget
`abandoned_plan_rate`	Created, 0 edits, 0 export in 24h	Abandoned ≠ accepted
`dsp_share_edited_down_rate`	% of plans where Strategist lowered a DSP `spendPct`	**Smoking gun for DSP-default creep that beat the regression suite**

Sprint 1 ships a single Next.js dashboard route reading Postgres, refreshed nightly. Weekly review meeting reads it. No real-time alerting yet.

20. CI Architecture (GitHub Actions)

Workflow: `.github/workflows/eval.yml`. Triggers: PRs touching `prompts/**`, `src/lib/extract.ts`, `src/lib/generatePlan.ts`, `src/lib/validate.ts`, `src/lib/types.ts`, `evals/**`, `model.config.json`; nightly 02:00 UTC against main; manual dispatch after model swap.

```
lint ─┐
      ├─> typecheck ─> deterministic_eval ─[passed]─> llm_judge_eval ─> comment_pr
unit ─┘                          │
                                 └─[failed]─> comment_pr (judge skipped)
```

Flaky LLM: Generator `temperature: 0.2`, single retry on API error only, none on content failure. Judge `temperature: 0`, three judges per (plan, brief), winner via 2-of-3 majority. Each brief runs independently.

PR comment shape:

```
KleverOne Eval — Prompt v0.4.1
Hard gates: 8/8 passed
Soft gates: 4/5 passed (rationale_quality 82% vs 85% baseline — flagged)
DSP share on pure-SSP set: 0% (5/5) ✓
Capability hallucinations: 0 ✓
Latency p95: 24s ✓
Full report: <artifact-link>
```

Baseline: last green main's `summary.json` mirrored to `evals/baseline/summary.json` on every green merge. No manual approval — green is green.

21. Ship Gates (the hard numbers)

Gate	Threshold	Why
Schema conformance	100%	Break = export + UI both fail
Capability hallucinations	0 across 20	Catalog static; non-zero = broken prompt
Deal hallucinations	0 across 20	Same logic with snapshot
DSP share on pure-SSP set	≤15% (PRD); **0% target**	The single regression the product exists to prevent
DSP justification non-empty on every DSP line	100%	A DSP line without justification IS the failure mode
p95 generation latency	≤25s end-to-end	5s buffer within AI engineer's 30s ceiling
`rationale_quality` win rate	≥45%	Below 45% = decisively worse than reference; 50% = parity
`allocation_rationale_quality` win rate	≥45%	The AC the Strategist edits against
`dsp_justification_adequacy` win rate	≥50%	DSP justification *is* the product — no regression accepted
"neither" rate on DSP-justification judge	0	Means counterfactual missing — core-pattern loss

PR cannot merge with any hard gate failing. Soft-gate regression requires Strategist approval on PR thread.

22. Failure Clustering and Triage

1. **Cluster.** Group failing briefs by the deterministic check tripped or the judge dimension that dropped. The cluster is the hypothesis.
2. **Drill.** Pull the 3 worst examples from `model_calls` (`is_eval_run=true, run_id=<failing>`). Read prompts, outputs, judge reasons.
3. **Diff.** Compare failing prompt version vs last green (git diff on `prompts/`). One commit usually owns the regression.
4. **Reproduce.** `npm run eval:single -- <brief-id>` against suspect commit and previous green. The diff in outputs is the bug.
5. **Fix or accept.** Revert/patch, or update reference (named-Strategist PR approval).
6. **Grow the suite.** If the regression slipped past existing briefs, the cluster's signature becomes a new brief. The suite grows on every miss.

PART D — CROSS-CUTTING

23. Auth, Identity, Secrets

v1: Auth.js (NextAuth v5) with Email/Magic-Link provider, backed by `app.users`. Email allowlist enforced in `signIn` callback. Sessions as JWTs in HTTP-only cookies. **Plus** Vercel "Vercel Authentication" deployment protection on production — the public URL itself requires Vercel team login before app code runs.

Alternative considered (Clerk): rejected. `app.users` is the load-bearing identity surface for every future agent — owning it locally is correct.

Path to OIDC (post-v1): Klever's IdP (likely Google Workspace; possibly Okta/Entra) lights up by adding the matching Auth.js provider and dropping email. Hours, not days, provided `users.email` stays the join key. Add `provider` + `provider_subject` when SSO ships.

Env vars (Vercel; never committed)

```
ANTHROPIC_API_KEY
DATABASE_URL                  # Vercel Postgres pooled
DATABASE_URL_UNPOOLED         # for migrations
BLOB_READ_WRITE_TOKEN         # Vercel Blob for deal snapshots
AUTH_SECRET                   # NextAuth JWT signing
AUTH_URL                      # production URL
EMAIL_SERVER                  # SMTP (Resend recommended)
EMAIL_FROM
ALLOWLIST_EMAILS              # comma-separated
SENTRY_DSN
SENTRY_AUTH_TOKEN             # source-map uploads at build
CRON_SECRET                   # validate Vercel Cron header
```

24. Observability, Logging, Prompt Versioning

Errors: `@sentry/nextjs` via `instrumentation.ts`. `release` set to git sha.

Model-call log: every Anthropic call writes one `model_calls` row inside the same transaction as the plan/brief it produced (or before, with `plan_id = null` for failed calls). Postgres is the right home, not Logflare — rows join to plans/users for debugging.

Confidentiality: Strategist briefs may contain client-confidential info. Treat `model_calls.request` as confidential; restrict the analytics Postgres role from reading it. Document a 90-day retention policy with a scheduled prune job (sprint 2).

Prompt versioning: semver + git sha + content hash, all in `prompt_versions`. Prompts live as TS template functions in `/src/lib/prompts/`. On boot, the pipeline renders each prompt against a canonical fixture, hashes it, and looks up or inserts the row. Semver bumps are manual; **hash drift without a bump is a CI failure**. Every plan and every model call carries `prompt_version_id` — any plan ever generated can be regenerated bit-for-bit.

25. Risks (Consolidated)

Risk	Layer	Mitigation
Catalog drift without a forcing function	Backend	Catalog owner named (PRD #2); weekly "catalog last-modified" Slack check from week 3
Snapshot vs live deal data divergence	Backend	Name snapshot `planning-deal-snapshot-*`; buying agent's freshness need is an explicit handoff problem
`brief_snapshot` denormalization mishandled	Backend	`getPlanInputBrief` helper; PR convention bans direct `plans → briefs` joins for input reads
Prompt regresses to DSP-default	AI / Eval	Hard gate on DSP share in CI
LLM hallucinates capability / deal refs	AI / Eval	Post-validation rejects; structured output forces resolution
Strategists distrust output, revert to slides	Frontend	Inline-editable plan + visible allocation rationale + edit telemetry
Schema needs to change once GAM shape lands	Backend / PRD	v1; reserve v2 migration; thin export layer (PRD #3)
Golden set staleness	Eval	Quarterly named-Strategist re-review; online edit rate as leading indicator
Judge drift	Eval	Pin judge model version; monthly kappa re-measurement; alert on drop >0.1
Deterministic checks insufficient	Eval	Top-10 most-edited fields classified quarterly → add check, tighten judge, or add brief

26. Composability — What Next Agents Inherit

Surface	Load-bearing because
`app.users` (email, id)	Every agent attributes work to a Strategist; SSO migration touches one table
`app.briefs`	Buying agent reads the same brief that produced the plan; reporting agent learns from extraction edits
`app.plans` + `app.plan_lines`	Buying agent's input. Already shaped against GAM line-item translatability. **PRD #3 must close** before this schema is v1-final, or buying agent inherits a translation layer
`app.plan_edits`	Optimization agent's training signal — highest-fidelity feedback the platform will ever get
`app.model_calls` + `app.prompt_versions`	Reproducibility, A/B prompt comparisons, regression debugging across all agents
Capabilities catalog	Shared vocabulary. One catalog, one source.
Deal snapshot (`planning-deal-snapshot-*`)	Planning-grade freshness only. Buying agent owns its own live path.

Sprint-1-only: email-magic-link auth (→ SSO), deal-snapshot seed JSON (→ MCP calls), hardcoded gate rules (→ data-driven months later).

27. Verification To-Dos (day 1 of sprint)

1. **Anthropic prompt-caching availability** for `claude-sonnet-4-6` on the team's account tier. If still beta-header territory, confirm header name and TTL.
2. **Next.js 16 streaming pattern.** Architecture uses a Route Handler because Server Functions return a single response per the docs at `node_modules/next/dist/docs/01-app/01-getting-started/07-mutating-data.md` and `15-route-handlers.md`. If Next 16 has added a streaming-action pattern, prefer it.

Neither blocks planning — both shift specific lines, not architecture.

28. Sprint 1 Execution Plan (suggested, 2 engineers × 10 days)

Day	AI engineer	Full-stack engineer
1	Lock `Brief` + `Plan` schemas; port `assessDspGate`; prompt v0 for extraction	Bootstrap, Auth.js + allowlist, Sentry, Postgres + Drizzle, `instrumentation.ts`
2	Extraction pipeline + retry + validator; first `model_calls` write	Route map, `AuthShell`, `/plans` with empty state
3	Generation prompt v0 with full catalog; tool-use `emit_plan`; partial-JSON streaming	`BriefInput` + Server Action; `/plans/new`
4	Post-validation (capability/deal resolve, share sums, justification rules)	`ExtractedBriefConfirmation` with RHF+zod; field-level edit capture
5	Wire `/api/plans/generate` SSE; first end-to-end happy path	`PlanWorkspace` skeleton; `PlanTable` columns + types + skeleton rows
6	Golden brief set v0 (10 briefs); deterministic checks (sums, resolve, threshold)	Streaming render; inline edit (Spend %, Rationale, Allocation); `logEdit`
7	Briefs to 20; remaining deterministic checks; CI workflow scaffold	`DspGateSummary`; `EditsCapturedIndicator`; `ExportMenu`
8	Judge rubrics v0; calibration set; pairwise harness	Error states (timeout, schema mismatch, stream drop); `error.tsx`
9	Tune prompts against eval; first PR-comment artifact	Polish: tabular figures, mono fonts, accessibility audit, focus order
10	Ship gates locked; documentation; Strategist sign-off on references	Internal walk-through; deploy; smoke test with pilot Strategist

Buffer: half a day each for the surprises that will appear in week two.

Critical Files for Implementation

- `planning-agent-mock/src/lib/types.ts` — contract starting point; add `path`, `share_ceiling`, `share_ceiling_reason` on `PlanLine` and a `BriefEdit` type
- `planning-agent-mock/src/lib/generatePlan.ts` — port `assessDspGate` verbatim as the deterministic stage-2 gate; LLM call replaces the hand-coded plan bodies
- `planning-agent-mock/src/lib/validate.ts` — extend with capability/deal-ref resolution, ceiling enforcement, justification-presence rules
- `planning-agent-mock/src/lib/catalog.ts` — move catalog out to JSON; rewrite as typed loader + helpers
- `planning-agent-mock/src/components/PlanTable.tsx` — preserve the dedicated-justification-column pattern; extend with edit affordances

- `planning-agent-mock/src/components/ConfirmationView.tsx` — replace with RHF + zod; per-field edit capture
- Next.js 16 docs: `node_modules/next/dist/docs/01-app/01-getting-started/07-mutating-data.md` and `15-route-handlers.md`